FACEATTEND · COMPLETE VIVA HANDBOOK

Face Recognition Attendance System

Everything you need for the viva in one place: what the project is, how it is built, how every module works, the AI models, security, database, API, how to run and demo it, and quick revision.

1. Project at a glance
2. Technology stack
3. System architecture
4. Folder structure
5. Modules and what they do
6. Registration workflow
7. Attendance (kiosk) workflow
8. AI models & transfer learning
9. Face matching algorithm
10. Anti-spoofing (liveness)
11. Security
12. Database design
13. API endpoints
14. Settings and numbers
15. How to run
16. Demo script for the viva
17. Testing
18. Advantages, limitations, future scope
19. Key terms (glossary)
20. Last-minute revision

Companion PDFs in `docs/`: `FaceAttend-Viva-Questions.pdf` (50 Q&A), `FaceAttend-Transfer-Learning.pdf` (models in depth), `FaceAttend-Documentation.pdf` (full technical documentation).

1 Project at a glance

Item	Detail
Title	**FaceAttend: Face Recognition Attendance System with Anti-Spoofing**
Type	Full-stack web application (React + Node.js/Express + MongoDB) with AI face models
Problem	Manual roll-call is slow, error-prone and allows proxy attendance. Simple face systems can be fooled by a photo and may mark the wrong person.
Solution	A kiosk camera recognises each person's face, checks they are a real live person, and marks attendance once per day. An admin console manages users and reports.
Key idea	Pre-trained deep-learning models (transfer learning) turn each face into 128 numbers; faces are compared by distance.
Top priority	**Never mark the wrong person.** If unsure, show "Unable to verify identity".

Objectives

1. Mark attendance automatically and contactlessly using face recognition.
2. Prevent proxy attendance with anti-spoofing (photo, video and screen attacks).
3. Avoid false matches with strict thresholds, a margin check and multi-frame confirmation.
4. Allow only one attendance record per person per day.
5. Protect biometric data with encryption and secure admin login.
6. Give the admin a dashboard, a searchable attendance list and Excel export.

Main features

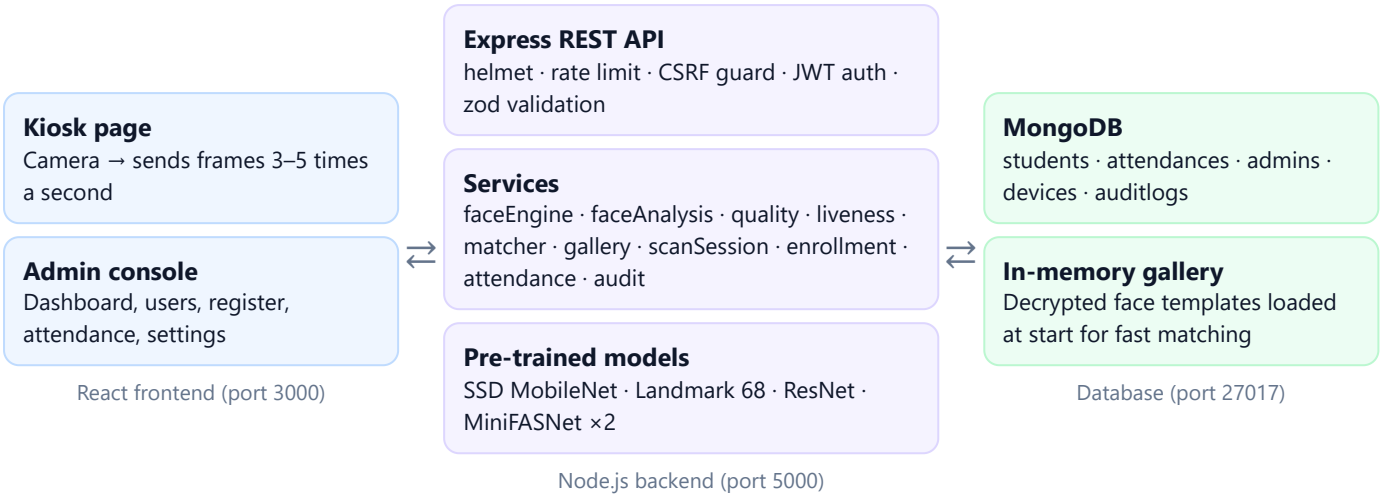
- Guided 5-pose face registration with automatic capture
- Duplicate-face detection at registration
- Kiosk scanning with live on-screen hints
- Passive liveness (2 models) + head-turn challenge
- Present / Late status by time zone
- Dashboard: present, late and absent today, last 7 days
- Attendance filters, status edit, delete, Excel export
- Kiosk device activation and deactivation
- Audit log of every important action
- Encrypted face templates, no photos stored

2 Technology stack

Layer	Technology	Why it is used
Frontend	React 19, React Router 7	Single-page app for the admin console and the kiosk screen; camera access through the browser
Backend	Node.js 22, Express 5	REST API; runs all face processing on the server
Database	MongoDB + Mongoose 8	Stores users, encrypted face samples, attendance, devices and the audit log
Face AI	@vladmandic/face-api 1.7 on TensorFlow.js 4.22 (WASM backend)	Face detection, 68 landmarks, 128-number face descriptor
Anti-spoof	onnxruntime-node + MiniFASNet (ONNX)	Real vs. fake face score
Security	helmet, bcryptjs, jsonwebtoken, express-rate-limit, zod, Node crypto (AES-256-GCM)	Secure headers, password hashing, sessions, rate limits, input validation, encryption
Uploads	multer (memory), jpeg-js, pngjs	Receive camera frames safely without saving files to disk
Reports	ExcelJS	Attendance export to `.xlsx`
Testing	node:test + supertest; Jest + React Testing Library	58 backend tests and 9 frontend tests

Stack name to say: **MERN stack** (MongoDB, Express, React, Node.js) **+ TensorFlow.js + ONNX Runtime**.

3 System architecture



Why processing is on the server

- Face data and models are never sent to the browser.
- A user cannot tamper with the result in the browser.
- The kiosk can be any device with a camera and a browser.

Server start-up order (`server.js`)

1. Load and check configuration (`.env`)
2. Connect to MongoDB and run migrations
3. Load the 5 AI models
4. Decrypt face templates into the gallery
5. Start listening on port 5000

4 Folder structure

```
facedetection/
├── backend/
│   ├── server.js              start-up: config → DB → models → gallery → listen
│   ├── .env                   secrets (never committed)
│   ├── models/                pre-trained model files (face-api + antispoof/*.onnx)
│   ├── scripts/               create-admin, download-models, evaluate, evaluate-liveness, rotate-keys
│   ├── test/                  58 automated tests
│   └── src/
│       ├── app.js             Express app, security middleware, routes
│       ├── config/            all settings, validated with zod
│       ├── db.js, migrations/ database connection and safe upgrades
│       ├── models/            Mongoose schemas: Student, Attendance, Admin, Device, AuditLog
│       ├── middleware/        auth, CSRF guard, uploads, rate limits, validation, errors
│       ├── routes/            auth, kiosk, students, attendance, admin
│       ├── services/          face engine, liveness, matcher, scan session, enrollment ...
│       └── utils/             logger, errors, time zone, biometric encryption
├── frontend/
│   └── src/
│       ├── App.js             page routes
│       ├── api/client.js      fetch wrapper (cookies + CSRF header)
│       ├── auth/              login state (AuthContext)
│       ├── hooks/useCamera.js camera access
│       ├── components/        FaceCapture (5-pose capture), shared UI
│       └── pages/             Home, Login, Kiosk, admin/* (Dashboard, Students, Register ...)
└── docs/                      documentation and viva PDFs
```

5 Modules and what they do

Frontend pages

Page / file	URL	What it does
Home.js	/	Entry page with "Kiosk" and "Admin console" cards
Login.js	/login	Admin sign-in
Kiosk.js	/kiosk	Activate the device, camera scanning, hints, result screen
Dashboard.js	/admin	Active users, present / late / absent today, today's check-ins, last 7 days, warnings
Students.js	/admin/students	User list, search, filters for re-registration and duplicates
RegisterStudent.js	/admin/students/new	Registration wizard: details → 5 face poses → save
StudentDetail.js	/admin/students/:id	Edit, re-capture face, deactivate, delete
Attendance.js	/admin/attendance	Records with filters, status change, delete, Excel export
Settings.js	/admin/settings	Change password, kiosk devices, audit log
FaceCapture.js	(component)	Guides the 5 poses and captures automatically after 2 good frames

Backend services

Service	Job
faceEngine.js	Loads SSD MobileNet, Landmark 68 and ResNet; detects faces, finds landmarks and creates the 128-number descriptor
faceGeometry.js	Calculates head angle (yaw, roll, pitch) and eye openness from the landmarks
quality.js	Checks face size, sharpness, brightness and pose; returns a hint message
liveness.js	Runs the 2 MiniFASNet models (passive) and the head-turn challenge (active)
faceAnalysis.js	Combines all checks for one image into one result
matcher.js	Compares descriptors by Euclidean distance; applies threshold and margin rules
gallery.js	Keeps decrypted templates of all active users in memory
scanSession.js	Kiosk state machine: SEARCH → CHALLENGE → CONFIRM → DONE
enrollmentService.js	Validates the 5 samples, checks duplicates, encrypts and saves them
attendanceService.js	Marks attendance once per day, sets Present/Late, applies cooldown
audit.js	Writes the audit log
biometricCrypto.js	AES-256-GCM encryption and decryption of face templates

Backend routes

Route file	Responsibility
auth.js	Login, logout, session check, change password, kiosk activate and deactivate
kiosk.js	Receives camera frames and returns the scan result
students.js	Register, list, view, edit, re-capture face, delete, check duplicates
attendance.js	List, export Excel, change status, delete
admin.js	Dashboard data, devices, audit log, system info

6 Registration workflow

1. **Admin signs in** and opens *Register user*.
2. **Enters details**: name and roll number (the roll number must be unique).
3. **Captures 5 poses**: front, slight left, slight right, slight up and slight down. Each pose is captured automatically after 2 good frames.
4. **Each photo is checked** on the server (`/students/check-sample`): exactly one face, big enough, sharp, well lit, correct pose, and a real person (liveness).
5. **Descriptors are created**: the ResNet model turns each face into 128 numbers.
6. **Duplicate check**: if the face matches an existing user (distance < 0.42), the admin is warned.
7. **Encrypt and save**: the 5 descriptors are encrypted with AES-256-GCM and saved. **The photos are discarded.**
8. **Gallery updated**: the new user can scan at the kiosk right away. No training is needed.

Viva point: registration does **not** train the model. It only stores the face numbers.

7 Attendance (kiosk) workflow

Camera frame → Detect face → Landmarks → Quality check → Passive liveness → Descriptor → Match → Head-turn challenge → Confirm → Mark

Phase	What happens	Kiosk shows
SEARCH	Each frame: find the face, check quality, check passive liveness (≥ 0.6), create the descriptor and match it. The same person must be matched on **3 frames** (5 if the match is weaker).	"Look at the camera", "Move closer to the camera"...
CHALLENGE	A random task: turn the head slightly left or right within **10 seconds**. The same face must stay in view.	"Please turn your head slightly to your left."
CONFIRM	One more front-facing frame must match the same person.	"Look straight at the camera"
DONE	Attendance is saved once per day (Present or Late), with distance, confidence and liveness score.	"Attendance marked ✓" / "Already marked today"

Possible results

- **MARKED**: new record saved
- **ALREADY_MARKED**: already present today
- **UNVERIFIED**: not sure who it is, so nothing is marked
- **Liveness / challenge failed**: possible spoof, recorded in the audit log

Why so many steps?

- Multiple frames remove one-off wrong matches.
- The challenge defeats photos and still screens.
- The confirm step ensures it is the same person after the turn.

8 AI models & transfer learning

Transfer learning means reusing a model already trained on a huge dataset for a new task instead of training from zero. This project uses the **feature extraction** type: the models are **frozen** (not retrained) and only their outputs are used. **No training or fine-tuning is done.**

#	Model	Job (nickname)	Input → Output	Runs in
1	**SSD MobileNet v1**	Face detection (*Finder*)	Camera image → face box + score	faceEngine.js
2	**Face Landmark 68**	68 face points (*Pointer*)	Face crop → 68 (x, y) points	faceEngine.js, faceGeometry.js
3	**Face Recognition ResNet**	Face descriptor (*Fingerprinter*)	Aligned 150×150 face → 128 numbers	faceEngine.js → matcher.js
4	**MiniFASNetV2**	Anti-spoof (*Guard 1*)	80×80 crop (2.7× face) → real score	liveness.js
5	**MiniFASNetV1SE**	Anti-spoof (*Guard 2*)	80×80 crop (4.0× face) → real score	liveness.js

How each model is built (simple)

- **SSD** = Single Shot Detector: finds objects in one pass. **MobileNet** = a light, fast CNN.
- **ResNet** = Residual Network: a deep CNN with "skip connections" that make deep networks train well.
- **MiniFASNet** = a small CNN for Face Anti-Spoofing; "SE" adds a Squeeze-and-Excitation attention block.

Where the model files come from

- Models 1–3: the **@vladmandic/face-api** package (`npm run models` copies them into `backend/models`).
- Models 4–5: ONNX files from the open-source face-anti-spoofing project (Apache-2.0), checked with SHA-256.
- All are loaded once at server start.

Remember the order: Find → Point → Guard → Fingerprint (detection → landmarks → anti-spoof → recognition).

9 Face matching algorithm

Every face becomes a vector of 128 numbers. Two faces are compared using the **Euclidean distance**:

$$d(A, B) = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2 + \dots + (a_{128} - b_{128})^2}$$

A small distance means the same person, and a large distance means different people. Each user has 5 stored samples; the distance to a user is the **smallest** distance to any of their samples (strategy `min`).

#	Rule (matcher.js + scanSession.js)	Value
1	Best distance must be within the threshold	≤ **0.5**
2	The second-best *different* person must be at least this much further away (margin)	≥ **0.06**
3	The same person must be matched on consecutive good frames	**3** frames (5 if median distance > 0.4)
4	Confidence label stored with the record	**high** if ≤ 0.4, else **standard**
5	Otherwise the result is "unknown" (NO_MATCH or AMBIGUOUS)	"Unable to verify identity"

Why distance and not a classifier? A classifier must be retrained whenever a new person joins. Distance-based comparison works for any number of people, and a new user only needs to be registered.

10 Anti-spoofing (liveness)

1. Passive liveness (no action needed)

- Two MiniFASNet models look at the face at two zoom levels (2.7× and 4.0×).
- Each outputs probabilities (softmax); class 1 = "real".
- The two "real" scores are averaged: **score ≥ 0.6** passes.
- Detects print photos, phone and laptop screens, and replayed videos from texture, moiré and edge clues.

2. Active challenge (user action)

- A random instruction: turn the head slightly **left** or **right**.
- Head angle (yaw) is measured from the 68 landmarks; it must change by ≥ **0.15**.
- Must be done within **10 seconds**, with the same face kept in view.
- Blink detection is also available (eye aspect ratio), off by default.

Why both? A passive model alone can be fooled by a very good print; a challenge alone can be fooled by a video. Together they are much stronger. Every failure is saved in the audit log as a security event.

11 Security

Threat	Protection in this project
Stolen face data	Templates encrypted with **AES-256-GCM** (key in `BIOMETRIC_KEY`); **no photos stored**; templates are hidden from API responses
Password theft	**bcrypt** hashing with cost 12
Session hijack	**JWT** in an **httpOnly, SameSite=Strict** cookie (JavaScript cannot read it); separate admin and kiosk tokens; logout and password change invalidate old tokens
Brute-force login	20 attempts per 15 min per IP; account locked for 15 min after **5** wrong passwords
CSRF	Requests must carry the `X-Requested-With` header and an allowed Origin
XSS, clickjacking	**helmet** headers: Content-Security-Policy, X-Frame-Options, nosniff, HSTS
NoSQL injection, bad input	Every request validated with **zod** schemas; 20 KB JSON limit
Malicious uploads	Files kept in memory only; image type checked from magic bytes; size and resolution limits
Unauthorised kiosk	Kiosk must be activated by an admin; devices can be deactivated
Spoofing (photo, video)	Passive liveness + head-turn challenge
Wrong person marked	Threshold + margin + multi-frame + confirm step
Duplicate attendance	Unique database index on (student, day)
Secret leaks	Secrets in `.env` (not in Git); logger hides sensitive fields
Accountability	Audit log of logins, changes, exports and spoof attempts

12 Database design (MongoDB)

students

- `name`, `rollNo` (unique), `active`
- `samples[]` (hidden by default): `enc` (encrypted descriptor), `pose`, `quality`, `liveness`
- `sampleCount`, `createdAt`, `updatedAt`

attendances

- `studentId`, `studentName`, `rollNo`
- `dayKey` (YYYY-MM-DD), `checkInAt`, `status` (Present / Late)
- `source` (kiosk / manual / legacy), `deviceId`
- `verification`: distance, confidence, liveness, challenge
- **Unique index** (studentId, dayKey)

admins

- `username` (unique), `passwordHash` (bcrypt)
- `tokenVersion` (session invalidation), `failedLogins`, `lockUntil`, `lastLoginAt`

devices

- `name`, `active`, `createdBy`, `lastSeenAt`

auditlogs

- `action`, `category` (admin / auth / security / system)
- `actor`, `target`, `details`, `ip`, `createdAt`

Relationships: one student → many attendance records (one per day). One device → many attendance records. The audit log points to whichever record was changed.

13 API endpoints (all under `/api`)

Method	Path	Access	Purpose
GET	/health	Public	Server status
POST	/auth/login · /auth/logout	Public / Admin	Sign in, sign out
GET	/auth/me	Any	Who is signed in (admin / kiosk)
POST	/auth/change-password	Admin	Change password
POST	/auth/kiosk/activate · /deactivate	Admin / Kiosk	Turn this browser into a kiosk, or stop
POST	/kiosk/scan	Kiosk	Send a camera frame, get the scan step / result
POST	/students/check-sample	Admin	Check one registration photo
POST	/students	Admin	Register a user with 5 samples
GET	/students · /students/:id · /students/duplicates	Admin	List, view, find duplicate faces
PATCH · PUT · DELETE	/students/:id · /students/:id/face	Admin	Edit, re-capture face, delete
GET	/attendance · /attendance/export	Admin	List records, download Excel
PATCH · DELETE	/attendance/:id	Admin	Change status, delete a record
GET	/admin/dashboard · /admin/audit · /admin/system	Admin	Dashboard data, audit log, settings info
GET · PATCH	/admin/devices · /admin/devices/:id	Admin	List and (de)activate kiosk devices

14 Settings and numbers (`backend/.env`)

Setting	Value	Meaning
RECOGNITION_THRESHOLD	0.5	Maximum distance to accept a match
HIGH_CONFIDENCE_THRESHOLD	0.4	At or below this, confidence is "high"
RECOGNITION_MIN_MARGIN	0.06	Required gap to the second-best person
DUPLICATE_FACE_THRESHOLD	0.42	Duplicate warning at registration
STABLE_FRAMES	3	Frames that must agree
LIVENESS_THRESHOLD	0.6	Minimum "real" score
CHALLENGE_TURN_DELTA / TIMEOUT	0.15 / 10 s	Head turn required and time allowed
MIN_FACE_SIZE_PX / ENROLL_MIN_FACE_SIZE_PX	90 / 110	Minimum face size at kiosk / registration
MIN_SHARPNESS · MIN/MAX_BRIGHTNESS	20 · 55–205	Image quality limits
MAX_YAW · MAX_ROLL_DEG	0.35 · 20°	Maximum head turn / tilt for a normal scan
ATTENDANCE_COOLDOWN_SEC	60	Repeat scans of the same person within 60 s are answered from memory ("Already marked")
APP_TIMEZONE · LATE_AFTER	Asia/Kolkata · HH:MM	Day boundary and Late time
ADMIN_SESSION_HOURS · KIOSK_SESSION_DAYS	8 · 30	How long logins last
JWT_SECRET · BIOMETRIC_KEY	secret	Session signing key · face encryption key (back it up)

15 How to run

First time only

```
cd backend
npm install
npm run models
npm run create-admin -- het
cd ..\frontend
npm install
```

Every time (MongoDB must be running)

```
Terminal 1:  cd backend   →  npm start
Terminal 2:  cd frontend  →  npm start

Open  http://localhost:3000
Admin /admin  ·  Kiosk /kiosk
```

Other command (in `backend`)	Use
`npm test`	Run the 58 backend tests
`npm run evaluate -- <folder>`	Measure recognition accuracy and choose the threshold
`npm run evaluate-liveness -- --live <dir> --spoof <dir>`	Measure anti-spoof accuracy
`npm run rotate-keys`	Change secret keys safely
`npm run build` (in frontend)	Production build, then served by the backend on port 5000

16 Demo script for the viva (about 5 minutes)

1. **Start both servers** before the viva and open `http://localhost:3000`. Explain the two sides: kiosk and admin console.
2. **Sign in as admin (het)** → show the **Dashboard**: active users, present, late, absent, last 7 days.
3. **Register a user** → show the 5-pose capture, quality hints and automatic capture. Say: "Photos are not stored; only encrypted 128-number descriptors."
4. **Open the kiosk** (/kiosk) → look at the camera → follow the head-turn instruction → "Attendance marked ✓".
5. **Scan again** → "Already marked today" (one record per day).
6. **Show a photo on a phone** to the kiosk → it is rejected (liveness / challenge failed).
7. **Attendance page** → show the new record with its verification details → click **Export Excel**.
8. **Settings** → **Audit log** → show the login, registration and spoof attempt records.

Before the viva: check that MongoDB is running, the camera works in the browser, the room has good front lighting, and at least one user is registered. Keep a phone photo ready for the spoof demo.

17 Testing

Test file / type	What it checks
unit.recognition	Distance, threshold, margin, ambiguous matches, duplicate detection
unit.scanSession	SEARCH → CHALLENGE → CONFIRM → DONE flow, timeouts, wrong-person switches
unit.utils	Encryption, time zone day keys, log redaction
api.security	Login, lockout, CSRF, unauthorised access, input validation, headers
api.attendance	One record per day, status change, export, filters
integration.engine	Real models on real images: detection, descriptor, liveness
Frontend (Jest, 9 tests)	Pages render, login errors, redirects, dashboard, users list
End-to-end	Real browser with a fake camera: success, already marked, failed challenge, mobile layout

Result: **58/58 backend** and **9/9 frontend** tests pass. Tests use a separate database, so real data is never touched.

18 Advantages, limitations, future scope

Advantages

- Fast and contactless
- Stops proxy attendance
- No wrong-person marks by design
- No model training needed
- Add users any time
- Encrypted biometric data
- Runs on a normal PC, no GPU
- Excel reports and audit trail

Limitations

- Needs good lighting
- Identical twins can be confused
- High-quality 3D masks may pass
- Large changes (heavy beard, mask) may need re-registration
- Needs a camera at each kiosk
- Thresholds should be tuned on local data

Future scope

- Cloud deployment with HTTPS
- Mobile app
- SMS / email alerts to parents
- Classes, subjects, timetables
- Depth / infrared camera liveness
- GPU backend for many kiosks
- Mask-aware recognition

19 Key terms (glossary)

Term	Simple meaning
CNN	Convolutional Neural Network: a deep-learning model that learns patterns in images
Transfer learning	Reusing a model trained on a big dataset for a new task
Feature extraction	Using a frozen pre-trained model only to produce features (here, 128 numbers)
Fine-tuning	Retraining some layers on your own data (not done here)
Embedding / descriptor	The list of 128 numbers that represents a face
Euclidean distance	Straight-line distance between two vectors; small = similar
Threshold	The cut-off value for accepting a match (0.5)
False positive / negative	Wrong person accepted / right person rejected
Landmarks	Key points on the face (68 here)
Liveness / anti-spoofing	Checking that the face is a real live person
Softmax	Turns model outputs into probabilities that add up to 1
Yaw / roll / pitch	Head turn left-right / tilt sideways / nod up-down
ONNX	An open file format for AI models that runs on many platforms
TensorFlow.js / WASM	Google's ML library for JavaScript / a fast way to run it on any CPU
REST API	A way for the frontend to talk to the backend using HTTP URLs
JWT	JSON Web Token: a signed login token
bcrypt	A slow, salted password hashing method
AES-256-GCM	Strong encryption that also detects tampering
CSRF / XSS	Tricking a signed-in browser into a request / injecting malicious script
Kiosk	The camera screen at the entrance where people mark attendance

20 Last-minute revision

Find → Point → Guard → Fingerprint
SSD MobileNet → Landmark 68 → MiniFASNet ×2 → ResNet (128 numbers).

Freeze → Extract → Compare
Transfer learning by feature extraction: frozen models, 128 numbers, Euclidean distance.

Stores, doesn't train
Registration saves 5 encrypted descriptors; photos are discarded.

Numbers to remember

Models	**5**	Descriptor	**128** numbers	Landmarks	**68**
Registration poses	**5**	Match threshold	**≤ 0.5**	Margin	**0.06**
Duplicate	**0.42**	Stable frames	**3**	Liveness	**≥ 0.6**
Challenge	**10 s**, turn 0.15	Inputs	**150×150 / 80×80**	Encryption	**AES-256-GCM**
Password	**bcrypt 12**	Lockout	**5 → 15 min**	Tests	**58 + 9**
Ports	**3000 / 5000 / 27017**	Time zone	**Asia/Kolkata**	Admin user	**het**

60-second answer: "Explain your project."
"My project is a face recognition attendance system built on the MERN stack: React, Node.js with Express, and MongoDB. A kiosk camera sends frames to the server, where five pre-trained models are used through transfer learning, as feature extraction. SSD MobileNet finds the face, Face Landmark 68 marks 68 points, two MiniFASNet models check that the face is real, and a ResNet model turns the face into 128 numbers. These are compared with stored faces using Euclidean distance with a 0.5 threshold, a margin check and three stable frames, followed by a head-turn challenge, so a photo cannot be used and the wrong person is never marked. Attendance is saved once per day. Face data is encrypted with AES-256, and the admin can manage users, view a dashboard and export attendance to Excel."

If you don't know an answer: say what you do know about that part of the project, and mention where it is in the code (for example, "that logic is in matcher.js"). Never guess numbers; use the table above.